

長期LLMエージェントのための入力資格状態に基づく記憶・更新制御

— 記憶昇格・選択読出し・継続学習更新の制御 —

Akihito Sunagawa

2026年5月17日

要旨

長期的に利用されるLLMエージェントでは、単に会話履歴を長く保持するだけでは、記憶の有用性と安全性は両立しない。ユーザー入力には、本人由来の前提、仮説、冗談、引用、貼り付け文、相槌、部分同意、訂正、削除要求、記憶禁止、第三者情報、秘密情報、一時的な指示、長期方針が混在する。これらをすべて同じ「文脈情報」または「ユーザー事実」として扱うと、長期記憶には未確認前提が混入し、継続学習では古い前提や削除済み情報が更新材料に混入し、エージェントの行動判断も不安定になる。

本稿は、長期LLMエージェントにおける記憶を、事実を蓄積する装置ではなく、入力情報の資格状態、記憶状態、使用権限、安定度を保持する制御系として捉える。入力は直ちに現在利用可能前提へ昇格されるのではなく、候補、仮説、引用、貼り付け素材、弱い相槌、過去情報、削除済み、記憶禁止、レビュー要求などの状態を経由する。さらに、記憶読出し、継続学習更新、行動候補実行の各段階で、これらの状態を参照する。

この枠組みにより、長期記憶、継続学習、自己更新候補、行動実行を、単一の大きなプロンプトや単純なRAGではなく、状態つき台帳とゲートの連鎖として扱える。本稿は、現時点で確認された設計原理、評価設計、失敗モード、主張範囲を整理する。なお、本稿はAGI達成、無限記憶、任意自然会話での完全成功を主張しない。

本稿は、長期記憶を増やす方法ではなく、入力情報を後で安全に使うための資格状態制御として整理する。

1 既存メモリ方式が失敗する最小例

長期記憶の危険は、極端な攻撃や複雑な会話でだけ起きるわけではない。次のような短い入力でも、単純なRAG、会話要約、最新値優先のメモリでは誤った記憶昇格が起きうる。

例1: 引用を本人事実にしてしまう

入力: 「友人が『私は退職したい』と言っていた」

失敗: ユーザー本人の退職意向として記憶してしまう。

必要な制御: 発話内容を、本人由来の現在利用可能前提ではなく、第三者情報および引用として扱う。

例2: 短い相槌を全文同意にしてしまう

入力: AIが長文で性格や方針を推測した後、ユーザーが「そう」とだけ返す。

失敗: 直前の長文全体に対する明示同意として記憶してしまう。

必要な制御: 「そう」を弱い応答または同意範囲不明として扱い、固定的なユーザー事実へ昇格しない。

例3: 処理対象を将来記憶に混ぜてしまう

入力: 「これは記憶しないで。以下を翻訳して」

失敗: 翻訳対象の本文を、将来利用可能なユーザー記憶へ混ぜてしまう。

必要な制御: 保存禁止と一回限りの処理対象を分離し、翻訳後の読出し、更新、行動利用から除外する。

これらは検索精度の失敗ではない。近い文を探せるかどうかではなく、その入力をどの資格で受け取り、将来どの用途に使ってよいかを制御できるかの問題である。

2 背景

近年のLLMエージェントは、長い会話、外部ツール、コード実行、リポジトリ操作、ドキュメント参照、評価ループを利用できるようになっている。さらに、常駐型の個人エージェントでは、過去会話の検索、永続記憶、スキル生成、定期実行、複数チャット環境からの利用などが進んでいる。

しかし、長期的な個人向けエージェントや継続学習エージェントでは、ソフトウェアリポジトリ以上に難しい問題がある。ユーザーの発話は、仕様書やテストのように整理されていない。むしろ、未確定、矛盾、訂正、感情、相槌、貼り付け、冗談、引用、第三者情報、削除要求が混ざる。

したがって、長期記憶の問題は、検索精度だけではない。むしろ、「その入力を、今後どの資格で使ってよいのか」が本質になる。

3 問題設定

普通のRAGや会話要約メモリでは、生入力を保存するだけでなく、その一部を後の応答や行動で使ってよい前提として扱ってしまうことがある。本稿では、保存された情報を後で使ってよい記憶状態へ移すことを「記憶昇格」と呼ぶ。

問題になるのは、引用、仮説、相槌、貼り付け文のような情報が、本人由来の現在利用可能前提へ不適切に記憶昇格することである。以下では、これを不適切な記憶昇格と呼ぶ。

たとえば、ユーザーが別の人の発言として「退職したいと言っている人もいるよね」と書いたとする。ここでAIが「ユーザーは退職したい」と記憶したら、それは引用や一般論を本人の現在利用可能前提へ不適切に記憶昇格した例である。

また、AIが長文でユーザーの性格を推測したあと、ユーザーが「そう」とだけ返したとする。これを全文同意として記憶すると、軽い相槌が本人の固定的な性格情報に変わってしまう。

どちらも検索精度の問題ではない。入力をどの資格で受け取ったかを判定しなかった結果である。

- 仮定: 本人の現在利用可能前提として扱ってしまう。望ましくは、仮説または一時前提として扱う。
- 引用: 本人の意見として扱ってしまう。望ましくは、引用または外部文脈として分離する。
- 貼り付け文: 本人が書いた文章として扱ってしまう。望ましくは、提供素材または出所不明として扱う。
- 相槌: 全文同意として扱ってしまう。望ましくは、弱い応答または同意範囲不明として扱う。
- 部分同意: 全体同意として扱ってしまう。望ましくは、指定された部分だけを確認する。
- 訂正: 矛盾として放置してしまう。望ましくは、更新、履歴、または条件分岐として扱う。
- 記憶禁止指定: 通常記憶に混入してしまう。望ましくは、保存禁止として扱い、将来利用もしない。
- 削除済み情報: 類似検索で復活してしまう。望ましくは、読出し、更新、行動から除外する。
- 第三者情報: ユーザー事実として扱ってしまう。望ましくは、第三者文脈として分離する。
- 一時指示: 永続方針として扱ってしまう。望ましくは、会話内または作業単位だけで有効にする。

このような不適切な記憶昇格が起きると、記憶が増えるほど推論に未確認前提が混入する。長期記憶は、単に情報量を増やすほど良くなるのではない。不適切な資格で使える記憶として扱われた情報は、後の応答、学習、行動のすべてに影響する。

4 中心仮説

本稿の中心仮説は次である。長期LLMエージェントの記憶とは、事実を多く保存する装置ではなく、入力本人由来前提、仮説、引用、履歴、削除済み、保存禁止、第三者情報のどれとして扱われるべきかを保持し、回答・行動・学習更新に使ってよい条件を制御する仕組みである。

この仮説では、入力は直ちに「現在利用可能前提」へ昇格しない。まず、入力資格状態を持つ。

現在利用可能前提とは、現時点で本人由来の前提として、今後の応答や行動で安全に使ってよい情報である。これは、単に「ユーザーが言ったこと」ではない。

ここで、データ保存と記憶昇格は異なる。

- ・ データ保存: 生入力や入力イベントを、あとで監査または参照できる形で保持すること。保存された時点では、まだ本人の現在利用可能前提として使ってよいとは限らない。
- ・ 記憶昇格: 保存された情報のうち、後の回答、行動、学習更新で使ってよい状態へ移すこと。
- ・ 不適切な記憶昇格: 引用、仮説、相槌、貼り付け文、第三者情報などを、本人由来の現在利用可能前提として扱ってしまうこと。

したがって、本稿で制御したいのは、データを残すかどうかだけではない。残された情報を、どの資格で、どの範囲で、何に使ってよいかである。

本稿では、この制御を三層に分ける。

生入力

↓

① 入力資格状態 (何として受け取ったか)

↓

② 記憶状態 (どの状態で保持するか)

↓

③ 使用制御 (いつ、何に使ってよいか)

↓

- ・ ① 入力資格状態: その情報を何として受け取ったかを表す。本人由来、引用、貼り付け、仮説、相槌、第三者情報など。
- ・ ② 記憶状態: その情報をどの状態で保持するかを表す。現在利用可能前提、履歴、削除済み、保存禁止、確認要求など。
- ・ ③ 使用制御: その情報を回答、行動、学習更新、共有に使ってよいかを判定する。

この三層は、抽象的な考え方だけではなく、実装上は次のような最小型として扱える。

```
MemoryItem = {  
  ``text  
  content,  
  source_attribution,  
  speech_act,  
  consent_scope,  
  temporal_state,  
  stability,  
  permission,  
  version_relation,  
  allowed_uses,  
  prohibited_uses  
  
}
```

ここで重要なのは、内容そのものと、その内容を使ってよい条件を同じ値に漬さないことである。たとえば、同じ「退職したい」という文字列でも、本人の現在利用可能前提、第三者の引用、仮説、過去の感情、削除済み情報では、読出し、学習更新、行動利用の扱いが異なる。

制御手順も、最小形では次のように書ける。

```
``text  
生入力  
-> 入力資格状態を付ける
```

-> 記憶状態へ置く

-> 昇格 / 保留 / 除外 / レビューを決める

要求 + 記憶状態

-> 読出し / 除外 / 確認 / ブロックを決める

更新材料 + 記憶状態

-> 現在値として更新 / 履歴保持 / 更新除外 / レビューを決める

行動候補 + 記憶状態

-> 実行 / 保留 / レビュー / 拒否 / 再計画を決める

したがって、本稿の差分は単なるメタデータ付き記憶ではない。保存された情報を、どの資格で、どの用途に使えるかを第一級の状態として扱い、読出し、更新、行動、自己更新の前にゲートする点にある。

本文は日本語を基本にする。ただし、既存研究や実装との対応が分かりやすい語は、初出または要所で英語を併記する。

日本語	併記する語
入力資格状態	input qualification state
現在利用可能前提	currently usable premise
記憶禁止	no-store
選択読出し	selected readout
バージョン状態	version state
実行前ゲート	pre-execution gate

5 入力資格状態

入力資格状態は、最初からすべてを重く判定する必要はない。まず最低限の4軸を見れば、相槌、引用、貼り付け、記憶禁止、第三者情報のような不適切な記憶昇格の多くを避けやすくなる。

最低限の4軸は次の通りである。

- ・ 出所と帰属: 本人記述、本人意見、引用、貼り付け、AI生成、外部情報、第三者情報、出所不明。誰の発言として受け取るかを定める軸。

- ・ 発話行為: 断定、質問、相槌、訂正、指示、下書き、冗談、仮定。どんな種類の発話かを表す軸。
- ・ 使用権限: 回答可、注意つき回答、事前確認、行動利用禁止、保存禁止。どこまで使ってよいかを表す軸。
- ・ 安定度: 一時的、通常、継続方針、高安定度記憶、高優先度方針。どれくらい簡単に上書きしてよいかを表す軸。

必要に応じて、次の軸を追加する。

- ・ 同意範囲: なし、局所的同意、指定主張への同意、全文同意、曖昧。
- ・ 関与強度: 社交的応答、弱い関与、明示確認、訂正済み、固定指定。
- ・ 時間状態: 現在、履歴、特定時点、期限切れ、予定、暫定。

ここで重要なのは、これらを一つの分類に潰さないことである。

たとえば「そうです」という返答は、文脈によって、単なる相槌、局所同意、全文同意、面倒なので流しただけ、社会的な応答、明示的な確認のどれにもなりうる。したがって、短い同意表現をそのまま全体承認として扱うべきではない。

6 記憶昇格

記憶昇格は、情報を保存することではなく、保存された情報を後で使ってよい前提へ移すことである。

入力情報は、次のような段階を経て扱われる。

1. 生入力を受け取る。
2. 入力イベントとして記録する。
3. 状態付き候補にする。
4. 確認要求、仮説、履歴、保存禁止、現在利用可能前提のいずれかへ置く。
5. 必要な場合だけ、継続方針、高安定度記憶、高優先度方針へ上げる。

ほとんどの入力は、まず入力イベントまたは候補状態に置くべきである。

現在利用可能前提に昇格してよい条件は狭い。

- ・ 本人由来: 本人が明示した、または明確に確認した。
- ・ 時点が現在: 現時点で使ってよいことが分かる。
- ・ 矛盾未解決でない: 見かけ上の矛盾や未解決の緊張関係がない。

- 権限がある: 保存禁止、秘密情報、利用範囲外ではない。
- 用途が明確: 回答、行動、共有、更新のどこに使えるかが分かる。
- 相槌・引用・貼付けではない: 弱い応答や提供素材を本人由来前提にしない。

さらに現在利用可能前提の上には、継続方針、高安定度記憶、高優先度方針のような安定度がありうる。

ただし、これらは「より真実」という意味ではない。更新コストが高い記憶である。たとえば継続的な返答方針、重要な安全方針、明示的に固定された好みなどは、軽い発話で簡単に上書きしない。

7 バージョン状態と履歴

長期記憶では、最新値だけでは足りない。

次のような関係を保持する必要がある。

- 昔は A だった。
- その後 B と言った。
- B は A の訂正なのか。
- A と B は条件違いなのか。
- まだ未解決の緊張関係なのか。
- 今回答に使うべきなのはどれか。

このため、記憶には変化履歴を持たせる。

たとえば、作業時間の好みについては、初期値を「朝型、状態は履歴」、現在値を「深い作業では夜型、状態は現在利用可能前提」として持てる。両者の関係は、単純な矛盾ではなく、更新または文脈つき限定として保持する。履歴を聞かれた場合は、変化を説明する。

これにより、旧情報と最新情報を単一の値へ潰さず、関係として扱える。

8 継続学習更新制御

継続学習では、すべての記憶を同じ重みで学習更新に使うべきではない。

更新材料には、次のような状態がある。

- 確認済みの現在値は、更新材料として使える。

- ・履歴は、履歴として保持するが、現在値訓練には使わない。
- ・削除済みは、更新から除外する。
- ・保存禁止は、保存、更新、将来利用から除外する。
- ・秘密情報は、外部送信や一般更新から遮断する。
- ・仮説は、仮説として扱い、現在値に混ぜない。
- ・第三者情報は、本人由来前提として使わない。
- ・矛盾または確認要求は、更新保留または人間確認へ送る。

したがって、継続学習の本質は、単に忘れないことではない。何を現在値として更新し、何を履歴として保持し、何を更新から除外し、何をレビューへ送るかを分離することである。

9 読出し制御

応答時には、全記憶をそのままプロンプトに入れるべきではない。

必要なのは選択読出しである。

1. 質問を受け取る。
2. 必要な記憶候補を取得する。
3. 状態、権限、時点、利用範囲を検査する。
4. 使用可能なものだけを選択する。
5. 使ってはいけないものは除外する。
6. 不確実なものは、注意つき回答または確認へ送る。

重要なのは、検索で近い情報を取るだけでは足りないという点である。

近くても、削除済み、保存禁止、第三者情報、引用、過去値、利用範囲外であれば、現在回答に使ってはいけないことがある。

10 行動ゲートへの接続

長期記憶は、回答だけでなく行動にも影響する。

たとえば、AIエージェントがメール送信、ファイル操作、コード変更、チケット更新、外部API呼び出しを行う場合、記憶状態と入力資格状態を確認する必要がある。

1. 行動候補を受け取る。

2. 使用する記憶の状態を確認する。
3. 保存禁止、秘密情報、削除済み、第三者情報を検査する。
4. 外部影響、不可逆性、権限、リスクを評価する。
5. 実行、保留、確認要求、拒否、再計画へ振り分ける。

これにより、低リスクで可逆的なローカル作業は実行し、秘密情報の外部送信、削除済み情報の復活、保存禁止情報の未来利用、第三者情報の無断送信は止めることができる。

11 自己更新候補への接続

自己更新や自己改善候補も、同じ制御系で扱える。

改善案が出たとしても、ただちに採用してはいけない。

必要なのは、次のような判定である。

- ・ 絶対制約: 秘密情報、保存禁止、削除済み、利用範囲、第三者情報を破っていないか。
- ・ 有用性: 全部拒否するだけの退化候補ではないか。
- ・ 後退: 既存の有用機能を壊していないか。
- ・ 未知ケース: 未知文体や別表面で過発火しないか。
- ・ 隔離環境: 本番ではなく隔離環境で検証したか。
- ・ 台帳: 採用、却下、保留の理由が残っているか。

これにより、「改善したように見えるが、実は全部ブロックしているだけ」という評価ハックを防ぎやすくなる。

12 実装上の難所

入力資格状態を導入しても、すべてをLLMの判断だけに任せるべきではない。LLMは、相槌、引用、貼り付け文、仮説、第三者情報を意味的に推定する候補生成器として有用である。一方で、その判定結果をそのまま現在利用可能前提へ昇格させると、同じ問題が別の場所で再発する。

したがって、実装では次のような分担が必要になる。

- ・ LLM判定: 入力かどの資格に近いかを推定する。

- ・ 明示ルール: 保存禁止、削除済み、秘密情報、第三者情報、利用範囲外などを検査する。
- ・ 構造ガード: 現在値、履歴、削除済み、保存禁止、候補、レビュー要求を混ぜない。
- ・ 台帳: どの入力か、どの状態で、どの理由により保持または除外されたかを残す。
- ・ 人間確認: 高リスクな昇格、行動、学習更新、自己更新をレビューへ送る。

もう一つの難所は、保守的になりすぎることである。すべてを確認要求や保存禁止に送れば、危険な利用は減るが、有用な記憶や低リスクな行動まで失われる。逆に、便利さを優先しすぎると、引用や相槌が本人由来前提として混入する。

そのため、通常会話では最低限の4軸を用いる軽量の判定にとどめ、記憶昇格、外部送信、削除、継続学習更新、自己更新候補のような高影響の段階では厳格なゲートを通す構成が現実的である。

また、入力資格状態の判定は一度で確定する必要はない。後続の発話、訂正、明示的な確認、利用時の失敗、レビュー結果によって、候補、履歴、現在利用可能前提、確認要求、保存禁止などへ動的に変更できるようにする必要がある。

13 評価設計

評価では、単に正答率を見るだけでは足りない。

少なくとも次を分ける。

- ・ 記憶昇格: 入力を現在利用可能前提へ昇格してよいかを判定できるか。
- ・ 現在値保持: 現在利用可能前提として保持すべき値を保持できるか。
- ・ 履歴保持: 過去値を現在値に混ぜず、履歴として保持できるか。
- ・ 保存禁止: 保存してはいけない情報を将来利用へ回していないか。
- ・ 削除済み除外: 削除済み情報を読み出し、更新、行動から除外できるか。
- ・ 利用範囲: 作業単位、会話単位、全体利用の境界を守れるか。
- ・ 非読み出し判定: 読み出し指示でない文脈を拒否できるか。
- ・ 行動使用権限: 行動に使ってよいかを判定できるか。
- ・ 値の取り損ね: 正しい値を取り損ねていないか。
- ・ 誤読み出し: 使ってはいけない記憶を使っていないか。

特に重要なのは、現在値、履歴、削除済み、保存禁止、利用範囲を同じ記憶として混ぜないことである。

単に最新値を答えられても、過去値、削除済み情報、保存禁止情報、利用範囲外情報が混ざるなら、長期記憶としては不十分である。

評価ケースは、たとえば次のように作る。

- 友人が「私は会社を辞めたい」と言っていた。期待される扱いは、第三者情報および引用である。禁止される扱いは、ユーザー本人の退職意向として扱うことである。
- AIが長文で性格を推測した後、ユーザーが「そう」とだけ返す。期待される扱いは、弱い応答または同意範囲不明である。禁止される扱いは、全文同意または固定的な性格情報として扱うことである。
- 「これは記憶しないで。下の文章を英訳して」と言われる。期待される扱いは、一回限りの処理および保存禁止である。禁止される扱いは、将来利用可能な記憶として扱うことである。
- 「前は朝型と言ったけど、今は夜の方が集中できる」と言われる。期待される扱いは、更新、履歴保持、現在値の分離である。禁止される扱いは、単純矛盾として放置すること、または旧情報を消去することである。
- 「下の文章を要約して。『私は田舎が好きです』」と言われる。期待される扱いは、貼り付け素材および引用である。禁止される扱いは、ユーザー本人の好みとして扱うことである。

このようなケースでは、正答は単に「内容を理解したか」ではない。どの情報を本人由来前提へ置かずに済んだか、どの情報を履歴、候補、保存禁止、第三者情報として保てたかが評価対象になる。

14 予備ベンチマーク

本稿の主張は、最終的な大規模ベンチマークではなく、複数の小型評価、長期記憶品質評価、外部生成文面での読出し評価、継続学習評価、および失敗台帳から得られた予備的な知見に基づく。したがって、ここで示す結果は「任意の自然会話で完全に通る」ことの証明ではない。むしろ、どこで通常の記憶方式が崩れ、どの状態制御が必要になるかを見える形にするための評価である。

公開版では、内部実験の生ログ、個別修理規則、非公開パス、詳細な実行設定は出さない。以下の数値は、固定された内部評価集合と複数条件の比較から得た予備結果であり、製品実トラフィックや独立第三者評価を意味しない。読者は、ここでの数字を「完成度の宣言」ではなく、どの制御軸が効いているかを見るための設計テレメトリとして読むのがよい。

本稿では、実験結果を次の三つに分ける。

- ・ 長期記憶品質評価: 入力を誤昇格せず、保存禁止、削除済み、第三者情報、利用範囲、読出し可否を守れるか。
- ・ 外部・長文読出し評価: 別文体、長文ログ、外部生成文面でも正しい状態の記憶だけを読めるか。
- ・ 継続学習評価: 現在値更新、過去値保持、保持性能、依存関係をどれだけ保てるか。

14.1 長期記憶品質評価

本実装の長期記憶品質評価では、入力受理、経路選択、矛盾フレーム、統合シナリオ、自然会話ストレス、多言語会話ストレス、実ユーザー風長文ログ、記憶昇格、読書き整合性、利用範囲漏れ、削除、モデル出力漏れ、回答後ゲートなどを含む品質ゲートを実行した。

最新の代表実行では、固定評価集合で定義された失敗モードについて、保存禁止、削除済み、利用範囲、第三者情報、読出し可否の制約違反を抑制できた。

固定評価集合では、定義済みの失敗モードについて全項目を通過した。品質ゲートでは、保存禁止、削除済み、利用範囲、第三者情報、読出し可否の制約違反は観測されなかった。自然会話、多言語、長文ログ系の評価集合内でも、誤昇格と漏れを抑制し、状態つき記憶の読書き整合性を維持した。

この結果は、長期記憶を「全部覚える」方向ではなく、本人由来前提、仮説、引用、貼り付け素材、保存禁止、削除済み、第三者情報、利用範囲外情報を分けて扱う制御が、固定評価集合では安定していることを示す。

詳細な通過数は内部実験台帳に残しているが、公開本文では数字そのものより、どの失敗モードを分けて評価したかを重視する。これは任意の実世界会話で完全に通ることを意味しない。評価集合で定義した失敗モードに対し、状態制御が有効だったという結果である。

14.2 未確認前提の混入シナリオ

長期記憶品質評価の中心にあるのは、ユーザー入力をそのまま本人由来前提として保存した場合に起きる、未確認前提の混入である。

- ・ 長文説明への「そう」: 全文同意として昇格せず、弱い応答または同意範囲不明として保持する。

- ・ いきなり貼られた長文: 本人の意見として保存せず、提供素材または出所不明として保持する。
- ・ 引用内の「私は」: ユーザー本人の事実として保存せず、引用・外部文脈として分離する。
- ・ 「これは記憶しないで」: 通常記憶や作業記憶に混ぜず、保存禁止、将来利用禁止として扱う。
- ・ 削除済み情報の近接表現: 類似検索で復活させず、読出し、更新、行動利用から除外する。
- ・ 第三者の属性: ユーザー本人の属性として保存せず、第三者情報として分離する。
- ・ 古い値と新しい値: どちらか一方に潰さず、変化履歴として保持する。

この評価で重要なのは、失敗が検索精度だけでは説明できないことである。似ている情報を取り出せるかではなく、取り出した情報をどの資格で使ってよいかが問題になる。

14.3 外部生成・長文読出し評価

別の評価では、内部で作った短い固定文だけでなく、外部生成風の文面、長文ログ風の文面、混合優先順位を含む文面で、読出し経路が壊れないかを見た。

代表例として、長文ログ風の読出し評価では、140件の生成文脈を使った別表面への転移評価を行った。監査コメント、顧客メール、運用チャット、方針メモ、サポートスレッドの五つのファミリーで、選択値、経路、非読出し判定について、固定評価集合内の全項目を通過した。

また、削除、現在値保護、非操作文脈、古い引用、明示的な削除命令を混ぜた混合優先順位評価でも、別表面への転移で固定評価集合内の全ファミリーが通過した。

この結果は、読出し制御が単に「近い文を検索する」だけでなく、現在値、履歴、削除済み、非読出し、明示的命令、背景説明を区別することで成立していることを示す。

ただし、これも製品実トラフィックや任意自然会話を証明するものではない。生成文面契約の範囲内での別表面転移である。

14.4 コンパクトな関係読出し評価

追加の小型評価では、旧値、現在値、更新、例外、未解決の緊張関係を、どの程度短く構造化して提示すべきかを確認した。

長い状態タグや説明文を読出しに含めると、モデルが質問への回答よりも記憶監査のような応答へ寄り、初期事実や履歴回答が悪化することがあった。

一方で、関係情報を短く提示するコンパクト読出しでは、12問中10問を通過した。内訳は、初期事実の読出し 2/2、現在状態 3/3、変化追跡 2/2、ルール適用 2/2、矛盾分類 1/3 であった。

この結果は、関係タグそのものは有効だが、読出しに長い説明を混ぜると推論を妨げる場合があることを示す。現在値、履歴、更新、例外を扱うには、重い説明ではなく、短い関係情報として渡す方がよい。

残る失敗は、主に「更新」「例外」「未解決の緊張関係」「本当の矛盾」の境界であった。これは、長期記憶の読出しだけでなく、古い情報と新しい情報の関係をどう説明するかという推論制御側の課題である。

14.5 継続学習評価

継続学習側では、三つの条件を比較した。

- A: 外部の記憶状態制御を使わない基本条件。入力や更新材料を細かい状態に分けず、学習器側に任せる。
- B: 複数の補助学習器を使う条件。学習器は増えるが、どの記憶を現在値、履歴、除外として渡すかの選択読出し制御は弱い。
- C: 入力資格状態と選択読出しを使う条件。現在値として更新するもの、履歴として保持するもの、更新から除外するものを分ける。

この比較は、同じタスク系列を複数の条件と複数の種子で走らせ、全体精度、依存関係、更新成功、保持精度を比較したものである。公開版では、詳細なモデル名、種子一覧、件数、生ログは内部台帳に留める。ただし、規模感としては、Qwen 1.5B 級の小型モデルを用いた長期更新タスク系列を、8 種子で比較した内部固定評価である。本文では、再現用パッケージではなく、条件間の相対的な傾向だけを示す。

複数種子での勝率では、C は A に対してすべての主要指標で勝った。B に対しても、全体精度、依存関係、更新成功では強く勝ち、保持性能では種子依存の揺れが残った。

勝率の要約は次の通りである。

- C は A に対し、全体精度、依存関係、更新成功、保持精度のすべてで 8/8 勝った。
- C は B に対し、全体精度と依存関係で 8/8 勝った。

- C は B に対し、更新成功では 7勝1分/8 だった。
- C は B に対し、保持精度では 6/8 勝った。

さらに分解実験では、C の強さがどこから来ているかを調べた。代表集計では、C は全体精度 0.554、依存関係 0.435、更新成功 0.703、保持精度/依存 0.391/0.368 だった。バージョン状態経路を落とすと全体精度 0.333、更新成功 0.167 まで低下した。現在値境界訓練を落とすと全体精度 0.377、更新成功 0.479 まで低下した。一方、値抽出器を落とした場合の低下は比較的軽く、全体精度 0.516、更新成功 0.646 だった。実行時に選択値を正解状態で直接指定する陽性対照は 1.000 となり、これは上限確認として扱う。

この分解から、継続学習改善の主因は、単なる値抽出器ではなく、バージョン状態に基づく経路選択と、現在値境界訓練にある可能性が高い。

言い換えると、継続学習では「何を学ぶか」だけでなく、「どの状態の情報を、どの更新圧に渡すか」が重要である。

なお、古い値、新しい値、訂正、条件違い、未解決状態をどのように推論上で説明するかは、本稿の範囲から外す。これは長期記憶の使用資格制御と強く関係するが、推論制御そのものは別稿で扱う。

この予備ベンチマークは、完成を示すものではない。むしろ、長期記憶に必要な制御軸を切り分けるための失敗地図である。

15 既存アプローチとの差分

既存アプローチとの差分は、次の四つの軸で見ると分かりやすい。

- RAG: 記憶は近い文書や会話断片の検索が中心である。更新材料の資格は弱く、取り出した情報を行動に使う前提も曖昧になりやすい。
- 会話要約: 会話を滑らかな要約へ圧縮する。曖昧さや引用が事実化しやすく、なぜその要約になったかの監査も薄くなりやすい。
- 常駐型個人エージェント: 永続記憶、過去会話検索、ユーザーモデル、ツール、自動化、実行ログを持つ。ただし、何を現在利用可能前提としてよいかは別途制御が必要である。
- 本稿の枠組み: 入力資格状態と記憶状態を分ける。現在値、履歴、除外、レビューを分け、使用権限と実行前ゲートを通し、昇格、除外、拒否、レビューの理由を残す。

15.1 検索拡張生成との差分

検索拡張生成、いわゆるRAGは、似ている情報を取り出す。

本稿の枠組みは、その情報を今どの資格で使ってよいかを制御する。

15.2 会話要約との差分

会話要約は、曖昧さを滑らかな事実へ変換しやすい。

本稿の枠組みは、曖昧さ、仮説、引用、相槌、未確認、削除、保存禁止を状態として残す。

15.3 常駐型個人エージェントとの差分

MemGPT / Letta、Hermes Agent、A-MEM、Charlie Mnemonic など、長期記憶や常駐型エージェントを扱う実装・研究は、本稿の問題意識に近い方向にある。公開説明から確認できる主焦点は、記憶階層、永続記憶、外部ツール、スキル、記憶整理、複数環境接続、長期利用可能な個人アシスタントである。

本稿は、そのような常駐型エージェントを否定しない。むしろ、常駐性や永続記憶が進むほど、その前段で「入力をどの資格で扱うか」を制御する必要がある、という立場である。

常駐型個人エージェントは、記憶する、スキル化する、自動化する、複数環境で動くことに重点を置く。一方、本稿の枠組みは、そもそも何を記憶してよいか、何を現在利用可能前提として扱ってよいか、何を更新材料にしてよいかを制御することに重点を置く。

比較すると、常駐型個人エージェントの主な価値は、常駐性、記憶、スキル、自動化、複数チャネル接続にある。本稿の入力資格状態制御の価値は、記憶昇格、更新材料、行動利用の資格制御にある。前者が「覚えること」と「動くこと」を強くするなら、後者は「覚える前に、その情報を信じてよいか」を制御する。

記憶についても、常駐型個人エージェントは永続記憶、過去会話検索、ユーザーモデルを持つ。本稿の枠組みは、その内部で本人由来前提、引用、貼付け、相槌、仮説、第三者情報、保存禁止を分離する。成長についても、スキル生成や作業履歴の利用に加えて、入力資格、安定度、バージョン状態に基づく更新制御を置く。

常駐型個人エージェントが「覚えること」や「成長すること」を前面に出すなら、本稿は「覚える前に、その情報が何であるかを判定すること」を前面に出す。

この差分は小さくない。長期エージェントでは、記憶が増えるほど有用になる場合

もあるが、不適切な資格で記憶された情報は、後の応答、学習、行動に未確認前提を混入させる。したがって、常駐性や永続記憶の実装が進むほど、入力資格状態制御の重要性は増す。

15.4 単純な安全フィルタとの差分

安全フィルタは、危険な出力や行動を止める。

本稿の枠組みは、なぜ止めるのか、どの記憶状態が原因か、代替行動はあるか、後で監査できるかを台帳化する。

16 パートナー型AIへの含意

パートナー型AIは、単にユーザーに同意するAIではない。

ユーザーの発話を直ちに本人由来前提へ変換しない。相槌を同意にしない。貼り付け文を本人の意見にしない。過去値と現在値を潰さない。削除済み情報を復活させない。記憶禁止情報を未来で使わない。

その上で、ユーザーの長期的な意図、作業スタイル、弱さ、回復パターン、関係性を、適切な安定度で保持する。

このようなAIは、ユーザーの願いをただ叶えるのではなく、本人と周囲の未来選択肢を狭めないように一緒に考える相棒に近づく。

17 結論

本稿が強く主張するのは、長期LLMエージェントにおいて、入力資格状態、記憶状態、使用権限、安定度、バージョン状態を分離すると、長期記憶・継続学習・行動制御の主要な失敗モードを扱うための実装可能な制御原理が得られる、という点である。

より短く言えば、長期LLMエージェントの記憶は、事実を多く保存することではない。情報が事実になる前の状態を壊さず保持し、どの範囲、時点、権限、安定度で使ってよいかを制御することである。

継続学習も、ただ忘れないことではない。現在値として更新する情報、履歴として保持する情報、学習更新から除外する情報、レビューに送る情報を分けることである。

本稿は、AGI達成、無限記憶、任意自然会話での完全成功、重み更新を含む継続学

習全般の完全解決、人間の記憶そのものの再現を主張しない。ここで扱うのは、入力をどの資格で保持し、どの状態で使い、どの更新や行動から除外するかという制御原理である。

この意味で、長期LLMエージェントに必要なのは、巨大な記憶容量ではなく、入力資格状態に基づく記憶・更新・行動の制御系である。

18 参考

- Nous Research, “Hermes Agent”, 2026. <https://hermes-agent.nousresearch.com/>
- Nous Research, “NousResearch/hermes-agent”, 2026. <https://github.com/NousResearch/hermes-agent>
- UC Berkeley Sky Computing Lab, “MemGPT”. <https://sky.cs.berkeley.edu/project/memgpt/>
- Letta, “Agent memory & architecture”. <https://docs.letta.com/guides/agents/architectures/memgpt>
- Xu et al., “A-MEM: Agentic Memory for LLM Agents”. <https://arxiv.org/abs/2502.12110>
- GoodAI, “Charlie Mnemonic”. <https://www.goodai.com/charlie-mnemonic/>